

An Automaton-Theoretic Representation of Difference Distribution Tables

Anonymous Submission

Abstract. The difference distribution table (DDT) is the central object of differential cryptanalysis, yet representing it as a dense $2^n \times 2^n$ matrix scales poorly: its size grows exponentially in the S-box width, and even when it fits in memory the cryptanalytic searches performed over it are each implemented as a bespoke scan. We argue that the natural abstraction for a DDT is not a matrix but a formal language. By recursively partitioning the table into quadrants, every cell is identified with a word over a quaternary alphabet, the non-zero entries form a regular language, and a quaternary analogue of multi-terminal decision-diagram reduction makes the resulting automaton canonical. Evaluated on the entire SageMath S-box library—288 S-boxes from $n = 3$ to 9—the representation reduces the number of stored states by a median of 5 to $12\times$, and by up to $273\times$ for highly structured S-boxes, with the automaton verified cell-by-cell against the dense table in every case. The compactness is only half of the story. Once the DDT is a regular language, any property expressible as a regular condition over the quadrant encoding becomes a small query automaton, and the matching entries are obtained by a joint traversal that never materialises the matrix. We give a uniform query calculus, validate seven query families against exhaustive search, and show that the same construction and calculus apply verbatim to the linear approximation and boomerang connectivity tables, unifying differential, linear, and boomerang queries under a single engine.

Keywords: Differential cryptanalysis · Difference distribution table · Decision diagrams · Finite automata · S-box.

1 Introduction

Differential cryptanalysis, introduced by Biham and Shamir [6], is one of the most influential statistical attacks against symmetric-key primitives. Its central insight is that the propagation of a chosen input difference through an iterated cipher is far from uniform, and that pairs of input and output differences that occur with high probability can be combined into multi-round trails that distinguish the cipher from a random permutation or recover key material. In the decades since its introduction, the technique has been refined into a broad family of variants, including truncated, higher-order, impossible, and boomerang differential attacks. All of them share the same underlying object: the difference distribution table of the cipher’s nonlinear components, which records the frequency with which each output difference is produced by each input difference.

The size of this table is also its main limitation. For an n -bit S-box, the DDT is a $2^n \times 2^n$ integer matrix, and its exponential growth in n rapidly exhausts memory budgets. An 8-bit S-box already requires 64 KiB; beyond $n \approx 15$ –16 the dense matrix can no longer be materialised on commodity machines. As cryptanalysts increasingly turn their attention to larger S-boxes, ARX components, and word-oriented permutations, this cost forces a recurring compromise between the expressiveness of differential analysis and the size of the primitives that can actually be analysed.

Compounding this storage problem is a more subtle access problem. Even when a DDT fits in memory, most cryptanalytic tasks do not need every cell at once: they need to extract the small subset of entries that satisfy some structural property. Examples are pervasive in the literature, including enumerating all impossible differentials of an S-box, listing every (a, b) that achieves the differential uniformity, isolating truncated differentials in which only a few input bits are active, identifying entries that satisfy a linear relation imposed by the surrounding cipher, and filtering transitions whose probability lies above a chosen threshold. In the standard matrix view, each of these queries is solved by an ad-hoc procedure, written anew for every analysis, and each shares the same unavoidable lower bound: a full or partial sweep of the table.

In this paper, we argue that the right abstraction for the DDT is not a matrix but a formal language. By recursively partitioning the table into quadrants, every cell is identified with a unique word over a quaternary alphabet, and the set of words that correspond to non-zero entries forms a regular language $L(D)$. This language is recognised by a finite automaton whose terminal states carry the differential multiplicities, and whose structure is canonical once a quaternary analogue of multi-terminal binary decision diagram reduction is applied. The resulting representation exploits both the general redundancy present in every DDT, such as the abundance of zero entries, and the specific redundancy resulting from the algebraic and combinatorial structure of typical S-boxes.

The compactness of this representation is, however, only one half of the contribution. The other half, and the focus of this work, is the querying capability that the language-theoretic view unlocks. Once the DDT is encoded as a regular language, any subset of entries definable by a regular condition over the alphabet $\Sigma = \{0, 1, 2, 3\}$ becomes a regular language in its own right, and can be intersected with $L(D)$ using standard automata-theoretic constructions. The disparate ad-hoc scans listed above all collapse into a single uniform recipe: write the property as a regular expression, compile it into a query automaton, intersect with the DDT automaton, and enumerate the resulting language. Truncated differentials, impossible differentials, fixed-bit differentials, entries of extremal probability, and many other targets that have historically required specialised code are recovered as one-line queries. Importantly, this also lifts queries from a per-cell cost to a cost proportional to the size of the (already compressed) intersection automaton, so the dense matrix never has to be enumerated even implicitly.

We make three contributions. First, we describe the construction and canonical reduction of the DDT automaton (Section 3) and evaluate it on the *entire* SageMath S-box library [17], 288 S-boxes spanning $n = 3$ to 9 (Section 4); every reported automaton is verified cell-by-cell against its dense table. Second, we develop the query calculus and validate seven query families against exhaustive search (Section 5). Third, we observe that nothing in the construction is specific to the DDT, and show that the linear approximation table (LAT) and boomerang connectivity table (BCT) are represented and queried by the same machinery, placing differential, linear, and boomerang analysis on a common footing (Section 6). Section 7 concludes. A self-checking artifact reproduces every measurement.¹

2 Preliminaries

This section establishes the concepts and notation used throughout, covering the relevant properties of the cryptanalytic tables, finite automata, and decision diagrams.

2.1 Differential cryptanalysis and its tables

Let $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be a vectorial Boolean function (an S-box). For an input difference a , the derivative of F in direction a is $D_a F(x) = F(x + a) + F(x)$; counting how often each output difference occurs gives the difference distribution table.

Definition 1 (Difference distribution table). *The DDT of F is the $2^n \times 2^m$ matrix $\text{DDT}_F[a, b] = |\{x \in \mathbb{F}_2^n : F(x) + F(x + a) = b\}|$, for $a \in \mathbb{F}_2^n$ and $b \in \mathbb{F}_2^m$.*

We assume $m = n$ throughout, as is the case for the permutation S-boxes that dominate practice, and return to the rectangular case in Section 7. The most studied statistic of the DDT is the differential uniformity, which upper-bounds an attacker’s per-round success probability and is therefore kept small by design.

Definition 2 (Differential uniformity). $\Delta(F) = \max_{a \neq 0, b \in \mathbb{F}_2^m} \text{DDT}_F[a, b]$.

Definition 3 (Differential spectrum). *The differential spectrum of F is the multiset $\{\text{DDT}_F[a, b] : a \neq 0\}$; we write ω_i for the number of non-trivial pairs (a, b) with $\text{DDT}_F[a, b] = i$.*

Zero entries of the DDT are the *impossible differentials*: a transition that cannot occur immediately discards a key guess reached during partial decryption, which is the backbone of impossible-differential cryptanalysis. Throughout, the trivial differential $(0, 0)$, whose value is always 2^n , is overridden to 0; it carries no cryptanalytic information, and removing it increases the sparsity of the table.

¹ Artifact anonymised for review.

Two further tables, defined here for use in Section 6, record the behaviour of F under the other two main attack families. The linear approximation table [14] $\text{LAT}_F[a, b] = |\{x : a \cdot x = b \cdot F(x)\}| - 2^{n-1}$ collects the correlations of linear approximations, with $\cdot$ the inner product over $\mathbb{F}_2$; its largest off-origin magnitude is the linearity. For a permutation F , the boomerang connectivity table [8] $\text{BCT}_F[a, b] = |\{x : F^{-1}(F(x)+b)+F^{-1}(F(x+a)+b) = a\}|$ makes the boomerang switch [18] exact, and its boomerang uniformity is $\beta(F) = \max_{a,b \neq 0} \text{BCT}_F[a, b]$. All three are square integer matrices indexed by (a, b) ; they differ only in how a cell is computed.

2.2 Finite automata and decision diagrams

A deterministic finite automaton (DFA) over an alphabet Σ is a tuple $(Q, \Sigma, \delta, q_0, A)$ with transition function $\delta : Q \times \Sigma \rightarrow Q$, initial state q_0 , and accepting set A , recognising the regular language $L = \{w : \delta^*(q_0, w) \in A\}$. Regular languages are closed under union, intersection, and complement; intersection is realised by the product construction, whose reachable part is what a joint traversal explores.

A binary decision diagram (BDD) [13,1] is a rooted directed acyclic graph whose internal nodes are labelled by a variable and have two children; evaluation follows the `low` or `high` child according to the bit read. Bryant [7] showed that fixing a total variable order and applying two reduction rules—merge isomorphic subgraphs, and delete a node whose children coincide—yields the reduced ordered BDD, which is canonical and minimal. A multi-terminal BDD (MTBDD) [10] relaxes the terminals from $\{0, 1\}$ to an arbitrary finite set, so that an integer-valued matrix indexed by Boolean variables is represented by a single canonical diagram. The correspondence between such diagrams and automata is classical [11]; it is what lets us treat the diagram as a language recogniser.

3 An Automaton-Theoretic Representation of DDTs

We propose a DDT representation based on automata theory, resting on the hypothesis that the derivatives of the functions used as S-boxes in real ciphers are not random, but possess significant structural regularities that can be exploited for compression. We first identify the redundancy that a dense matrix fails to exploit, then describe the construction of the automaton and its reduction.

3.1 Structural redundancy in DDTs

For an n -bit S-box, representing the DDT as a matrix requires 2^{2n} elements, which quickly becomes prohibitive both for storage and for in-memory analysis as n grows. The inefficiency of this representation is the result of two kinds of redundancy. The first is the *general* redundancy present in every DDT: because the number of distinct differential values is far smaller than the number of cells, many cells share a value, and at least half of all entries are zero. The second is

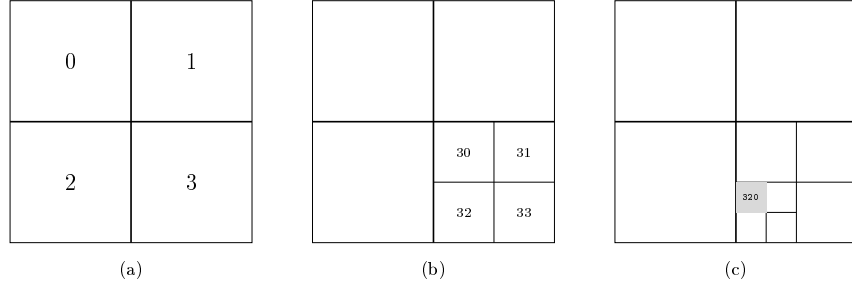


Fig. 1. Recursive quadrant encoding of an 8×8 DDT. (a) the table split into four quadrants, each a symbol of Σ ; (b) each quadrant split again, so sub-quadrants receive length-two words; (c) the highlighted cell encoded as 320.

The symbol read at each level encodes one bit of the row index a and one of the column index b . Writing the symbol as the pair (a_i, b_i) , we set

$$\sigma = 2a_i + b_i, \quad (1)$$

so that the input bit is the high bit and the output bit the low bit of σ , and the bits are read in the interleaved order $a_{n-1}, b_{n-1}, \dots, a_0, b_0$, most significant pair first. For instance, $a = 101_2$ and $b = 011_2$ give the bit pairs $(1, 0), (0, 1), (1, 1)$ and hence the word 213. This single convention is used everywhere below; in particular, an input-active position ($a_i = 1$) uses the symbols $\{2, 3\}$ and an output-active position ($b_i = 1$) uses $\{1, 3\}$.

Following this structural mapping, $L(D)$ is recognised by a quaternary automaton in which each path from the initial state to a terminal corresponds to a word $w \in \Sigma^n$: internal states are the sub-matrices produced by the recursive partition, and transitions are dictated by the symbols of Σ . Unlike a standard DFA, which outputs only accept or reject, this automaton must return the DDT values, so we equip it with a function $f_D : \Sigma^n \rightarrow \mathbb{N}$. Evaluating a word therefore both decides whether the differential is possible and returns its multiplicity.

Reduction. To store the automaton compactly, minimisation is necessary, but classical DFA minimisation cannot be applied directly: it would merge accepting states carrying different values. Instead we draw on the relationship between automata and decision diagrams [11] and on the integer terminals of the MTBDD [10], and reduce a quaternary variant of a decision diagram in which each internal node v has four children $E_0(v), \dots, E_3(v)$:

1. terminals are shared, one per distinct value, leaving a single dead terminal for 0;
2. if $E_0(v) = E_1(v) = E_2(v) = E_3(v)$, node v is redundant and is replaced by that common child;
3. if two internal nodes *at the same level* have identical children, they are merged.

Table 2. DDT of the 3-bit S-box of the SEA cipher (rows a , columns b). The S-box is APN, so every non-zero entry equals 2.

$a \backslash b$	000	001	010	011	100	101	110	111
000	0	0	0	0	0	0	0	0
001	0	2	0	2	0	2	0	2
010	0	2	2	0	0	2	2	0
011	0	0	2	2	0	0	2	2
100	0	0	0	0	2	2	2	2
101	0	2	0	2	2	0	2	0
110	0	2	2	0	2	0	0	2
111	0	0	2	2	2	2	0	0

The level qualifier in rule 3 reflects that a node’s identity in an ordered diagram is the pair (variable, children): two nodes with the same children at different depths are distinct, and merging them would corrupt evaluation. While the resulting automaton is not minimal in the ordinary sense, it is minimal and canonical in this ordered, multi-terminal setting, by Bryant’s argument [7] transported through the diagram–automaton correspondence [11]. The final step removes the redundant strings from the function table, leaving exactly one entry per distinct value.

A worked example. To illustrate the construction and its reduction, consider the 3-bit S-box of the SEA cipher, whose DDT is shown in Table 2. Because the S-box is APN, its only non-zero value is 2, and the trivial differential at $(0,0)$ is overridden to 0. The unreduced automaton spells out all 28 paths to non-zero cells. Reduction collapses the repeated structure: at the first level the branches on symbols 0, 1, and 2 lead to identically structured sub-matrices and are grouped, immediately sparing half the storage, and at the terminal level a single accepting state suffices for the value 2. The reduced automaton, shown in Figure 2, has nine states—eight internal and accepting states together with the dead terminal—and a function table with the single entry 2. We verified that it reproduces all 64 cells of the DDT exactly.

4 Results

We evaluated the representation on the SageMath cryptography library [17] (`sage.crypto.sboxes`), which gathers 288 S-boxes from the literature, all square, with widths from $n = 3$ to 9; of these, 283 are permutations. For each S-box we built the reduced automaton of its DDT and verified it cell-by-cell against the dense table computed by Sage, so every figure below is backed by an exact round-trip. We store the automaton as a contiguous array of states, each with four fields holding the indices of its successors; index 0 is the dead state, accepting states hold the differential value in place of successors, and the field width follows the state count. We report the *state ratio*, the dense cell count 2^{2n} divided by the number of automaton states.

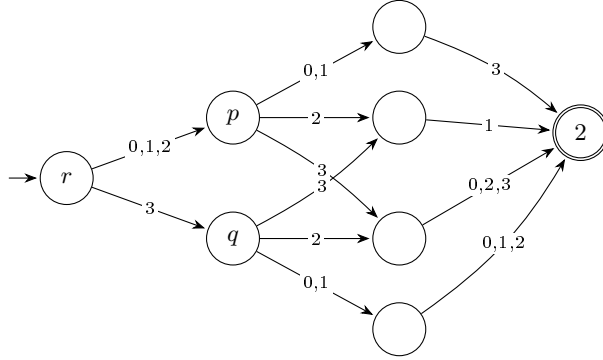


Fig. 2. Reduced automaton for the SEA DDT. Edges are labelled by the symbols taking each state to its successor; transitions to the dead state are omitted. The double circle is the single accepting state, of value 2.

Table 3. State ratio (dense cells / automaton states) over the SageMath S-box library, by width, as median [min-max]. The BCT is restricted to the 283 permutations.

n	#S-boxes	DDT	LAT	BCT
3	3	7.1 [4.9–9.1]	3.4 [3.2–5.3]	4.0 [3.4–4.9]
4	215	5.2 [4.6–8.8]	3.7 [3.1–5.7]	3.5 [2.9–9.1]
5	7	10.7 [9.6–11.2]	6.5 [6.1–6.6]	5.3 [4.8–10.0]
6	3	11.5 [10.7–11.5]	8.2 [3.9–9.5]	11.2 [10.0–11.2]
7	1	10.8	4.4	8.2
8	58	11.7 [11.5–273.1]	3.9 [3.4–69.7]	10.5 [8.5–125.1]
9	1	217.6	75.6	27.3

Across the library. Table 3 aggregates the state ratio by S-box size. Compression is present at every width and increases with n , as the larger tables offer more repeated structure to share. The spread at $n = 8$ is wide because the library mixes strong AES-like S-boxes with deliberately weak or highly structured ones; we return to the extremes below.

Named ciphers. Table 4 lists a cross-section of well-known S-boxes. The DDT compression grows with the differential uniformity: a larger Δ concentrates the entries into fewer distinct values and larger constant blocks, so the $\Delta = 64$ SKINNY 8-bit S-box (shared by Romulus, Remus, and ForkSkinny) is stored in 915 states (71.6 $\times$), and the deliberately weak CSS S-box ($\Delta = 128$) in only 240 states (273 $\times$). Translated into memory, the 64 KiB AES DDT becomes 42 KiB, the SKINNY table 7.2 KiB, and the CSS table under 1 KiB; for the 9-bit DryGASCON S-box the dense 256 KiB table is stored in 9.4 KiB. The byte-level gain is most pronounced exactly when the structural ratio is, since the field width is otherwise fixed by the state count. The full per-S-box results for all 288 S-boxes accompany the artifact.

Table 4. Reduced DDT automaton for selected S-boxes: state count and state ratio, with the LAT and BCT ratios of the same S-box (Section 6).

S-box	n	Δ	DDT states	DDT $\times$	LAT $\times$	BCT $\times$
SEA	3	2	9	7.1	3.4	4.0
Noekeon	4	4	30	8.5	4.3	7.5
PRESENT	4	4	39	6.6	4.7	5.3
SKINNY (4-bit)	4	4	35	7.3	3.7	5.8
PRINCE	4	4	48	5.3	3.4	3.6
GIFT	4	6	47	5.5	4.2	3.4
Midori S_{60}	4	4	52	4.9	3.5	3.6
Ascon	5	8	107	9.6	6.1	4.8
Fides (5-bit)	5	2	97	10.6	6.6	9.7
APN (6-bit)	6	2	356	11.5	9.5	11.2
WAGE	7	8	1516	10.8	4.4	8.2
AES	8	4	5393	12.2	3.8	12.0
Camellia	8	4	5386	12.2	3.8	12.0
CLEFIA S_0	8	10	5630	11.6	6.2	10.1
Skipjack	8	12	5652	11.6	3.8	10.3
Kuznyechik	8	8	5590	11.7	3.8	10.6
Fantomas	8	16	4556	14.4	9.1	8.5
SKINNY (8-bit)	8	64	915	71.6	23.0	29.6
CSS	8	128	240	273.1	69.7	125.1
DryGASCON256	9	128	1205	217.6	75.6	27.3

5 Applications

5.1 Computing differential properties

The representation supports the direct computation of standard differential metrics without ever generating the dense table. The differential uniformity is the largest value stored at a terminal, found by a single pass over the terminal set in $O(|T|)$ time, where $|T| \leq \Delta(F) + 1$. The differential spectrum is obtained by a depth-first traversal from the root that multiplies the number of valid transition symbols along each edge and, on reaching a terminal of value i , adds the accumulated path weight to the counter ω_i ; this uses $O(n)$ memory and time linear in the number of distinct paths in the reduced structure. Impossible differentials are exactly the words rejected by the automaton: a differential (a, b) is impossible iff its encoding is not in $L(D)$. For a single (a, b) this $O(n)$ evaluation is not advantageous over an $O(1)$ table lookup, but, as we now show, the language view pays off precisely when many entries are sought at once under a structural constraint.

5.2 Complex queries via automata operations

Once the DDT is a finite automaton, a cryptanalytic query becomes a regular *condition* over Σ , resolved by operations on languages. This turns the ad-hoc search for differential properties into a uniform, language-theoretic procedure:

1. formulate the query as a regular expression R over Σ ;
2. compile R into a finite automaton Q ;
3. compute the intersection of D and Q by a joint traversal (a lazy depth-first search over the two automata);
4. enumerate the resulting language, yielding the matching DDT entries.

Because regular languages are closed under union, intersection, and complement, queries compose: the complement $\overline{L(D)} \cap \Sigma^n$ is the impossible language, a value-restricted sub-automaton of D filters by probability, and products of query automata express conjunctions, all without leaving the framework. We write $a = a_{n-1} \cdots a_0$ and $b = b_{n-1} \cdots b_0$, use the encoding (1), and let $\cdot$ abbreviate (0|1|2|3).

Fixed input differences. To list all possible output differences for a fixed input $a = 0101$, constrain each position to the corresponding input bit and leave the output free, giving $R = (0|1)(2|3)(0|1)(2|3)$ (positions with $a_i = 0$ use $\{0, 1\}$, those with $a_i = 1$ use $\{2, 3\}$). This replaces the $O(2^n)$ row scan and is reused for other a by relabelling.

Truncated differentials. To require only that the two most significant input bits be active, $R = (2|3)(2|3) \dots$. Any mask in $\{0, 1, *\}$ on the input and output bits compiles to an automaton of $O(n)$ states; the active/inactive special case recovers the activity patterns underlying the MILP- and SAT-based active-S-box counts of Mouha et al. [16] and the bounds of ciphers such as SKINNY [2].

Impossible differentials. These are the words rejected by D . Intersecting a pattern automaton with $\overline{L(D)} \cap \Sigma^n$ enumerates the impossible cells of that pattern; e.g. for a 4-bit S-box with only the least significant input bit active, $R = (0|1)(0|1)(0|1)(2|3)$. This is the elementary step of impossible-differential cryptanalysis as first applied to Skipjack [5] and DEAL [12].

Extremal-probability differentials. Because each accepting state stores its value, filtering by probability is a one-pass restriction of D : keep only the accepting states with value $\geq \tau$ and prune. Then $L(D_{\geq \tau})$ are the differentials of probability $\geq \tau \cdot 2^{-n}$, and $\tau = \Delta(F)$ yields those attaining the uniformity, the seed of branch-and-bound trail search [15]. The construction visits each accepting state once, $O(|S|)$.

Hamming-weight bounded differentials. The words with $\text{wt}(a) + \text{wt}(b) \leq w$ are accepted by a counter automaton of $O(nw)$ states that adds each symbol's popcount (0 for 0, 1 for 1 and 2, 2 for 3). Single-bit-in, single-bit-out is $R = 0*2 \ 0*1 \ 0* \mid 0*1 \ 0*2 \ 0* \mid 0*3 \ 0*$, supporting the wide-trail verification of Daemen and Rijmen [9].

Table 5. Validated DDT queries. *res.* is the number of matching entries and *prod.* the product states visited; dense tables hold 256 and 65536 cells, with $|D| = 39$ and 5393. Every result equals the brute-force set.

query	PRESENT ($n=4$)		AES ($n=8$)	
	res.	prod.	res.	prod.
fixed input difference	7	18	127	148
truncated (2 MSB input active)	26	21	8128	1405
impossible (one input bit active)	12	11	129	135
extremal ($= \Delta$)	24	40	255	5395
Hamming $\text{wt}(a) + \text{wt}(b) \leq 2$	0	30	24	237
linear $a = b$	5	17	116	148
composite ($\geq \Delta/2$, $\text{wt} \leq 4$)	63	59	1091	1501

Linearly constrained differentials. The diagonal relation $a = b$ is $R = (0|3)^n$, since $\sigma \in \{0, 3\}$ exactly when $a_i = b_i$; any $\mathbb{F}_2$ -linear relation is accepted by a parity-tracking automaton with at most 2^r states for r constraints, extracting the entries compatible with the linear context imposed by the surrounding cipher.

Composite and comparative queries. Boolean closure combines these freely: high-probability low-weight differentials are $L(D_{\geq \Delta/2}) \cap L(\text{wt} \leq w)$, a single product. Queries also span two S-boxes: given D_F and D_G , the language $L(D_F) \setminus L(D_G)$ collects the differentials possible for F but impossible for G —a certificate of inequivalence useful when comparing candidate S-boxes during design.

For every query the running time is dominated by the size of the intersection, bounded by $|D| \cdot |Q|$ but usually far smaller, and the intersection is never materialised: a joint traversal touches only the compressed automaton and the small query automaton, so the memory savings of Section 4 carry over to query time.

5.3 Experimental validation

The original framework leaves these queries as constructions; we implemented them and checked each against ground truth. Table 5 reports, for the PRESENT ($n = 4$) and AES ($n = 8$) DDT automata, the number of matching entries and the number of product states the joint traversal visited, against the 256 and 65536 cells of the dense tables. For every query we also computed the answer by an exhaustive scan and confirmed set-equality; all seven families agreed exactly.

With one exception the traversal visits far fewer product states than the table has cells. The AES truncated query inspects 1405 states to enumerate 8128 matching entries against 65536 cells, and the structural queries stay in the low hundreds. The exception is the extremal query, whose value filter is inherently global and so scans essentially all of D ($5395 \approx |S|$), consistent with

Table 6. Validated LAT and BCT queries against the dense scan; “lin.” is the linearity and β the boomerang uniformity. Product states are reported against the 256 (PRESENT) and 65536 (AES) dense cells.

table query	S-box	res. prod.	
LAT $ \text{LAT} \geq \text{lin.}$	PRESENT (lin. 4)	36	57
LAT $ \text{LAT} \geq \text{lin.}$	AES (lin. 16)	1275	17101
BCT value = β	PRESENT ($\beta=16$)	2	49
BCT value = β	AES ($\beta=6$)	510	5483

its $O(|S|)$ bound and still independent of the dense matrix. The framework also surfaces structure for free: the empty PRESENT row for $\text{wt} \leq 2$ records that the PRESENT S-box admits no non-trivial differential with at most two active bits.

6 Beyond the DDT: the LAT and BCT

Nothing in Section 3 is specific to the difference distribution table: the construction takes any square integer matrix over the quadrant encoding. The linear approximation table and the boomerang connectivity table are therefore represented by exactly the same reduced automaton, with no change beyond supplying a different matrix. Their state ratios appear alongside the DDT in Tables 3 and 4. The LAT compresses less than the DDT, because its values span a range—the AES LAT takes 18 distinct values against the DDT’s 3, and the number of terminals bounds the diagram—whereas the BCT, with a small value set, compresses essentially like the DDT.

The query calculus generalises in the same way, turning linear and boomerang questions into the same primitive. High-bias linear approximations, which drive Matsui’s attack [14], are the magnitude filter $|\text{LAT}_F[a, b]| \geq \tau$ on the LAT automaton, with τ the linearity selecting the best approximations. Boomerang-uniformity cells, the differences that maximise the boomerang switch [18,8], are the value filter $\text{BCT}_F[a, b] = \beta(F)$ on the BCT automaton. We validated both on the PRESENT and AES tables against an exhaustive scan (Table 6); as with the extremal DDT query, these are global value filters and so scan their table automaton, which is nonetheless far smaller than—and is built without—the dense matrix. The upshot is that a single automaton type and a single query engine serve differential, linear, and boomerang cryptanalysis, with the property of interest residing entirely in the small query automaton.

7 Conclusion

We described an automaton-theoretic representation of difference distribution tables. After outlining the memory and access limitations of the dense matrix, we reframed the DDT as a formal language and represented it by a reduced

quaternary automaton that captures both the general and the S-box-specific redundancy of the table. Evaluated on the full SageMath S-box library, the representation reduces the number of stored states by a median of 5 to $12\times$, and by up to $273\times$ for highly structured S-boxes, without any loss of information.

Beyond the storage gain, the central observation is that the language-theoretic view turns the cryptanalytic searches performed over the table into operations on automata. The properties that can be phrased as regular conditions over the quadrant encoding of (a, b) —fixed-input and truncated differentials, impossible differentials, extremal-probability entries, low-weight transitions, linearly constrained pairs, and composite or cross-table queries—are all recovered through a single joint traversal of the DDT automaton and a small query automaton, and we validated each against exhaustive search. Because the construction is agnostic to how a cell is computed, the same machinery serves the linear approximation and boomerang connectivity tables, placing the three main attack families on a common representation.

Several directions extend this work. The square-table assumption can be lifted to rectangular $2^n \times 2^m$ tables by an uneven recursion that consumes input and output bits at different rates. The query class can be enriched from regular to context-free constraints, such as bounded-difference balances across rounds, by replacing the query automaton with a pushdown automaton and adapting the joint traversal. The same automaton can also serve as a generator of linear inequalities for MILP-based trail search, and quantifying the trade-off between automaton size and the number of generated inequalities is left to future work. Finally, the size of the reduced automaton appears to capture an intrinsic notion of the structural regularity of an S-box, and relating this size-based metric to differential properties is a natural analytical follow-up.

References

1. Akers, S.B.: Binary decision diagrams. *IEEE Trans. Comput.* C-27(6), 509–516 (1978)
2. Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: *CRYPTO 2016*. LNCS, vol. 9815, pp. 123–153. Springer (2016)
3. Berstel, J., Nait Abdallah, A.: Quadrees generated by finite automata. *AFCET* 61/62, 167–175 (1989)
4. Berstel, J., Morcrette, M.: Compact representation of patterns by finite automata. In: *PIXIM 89*, pp. 387–402 (1989)
5. Biham, E., Biryukov, A., Shamir, A.: Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. In: *EUROCRYPT 1999*. LNCS, vol. 1592, pp. 12–23. Springer (1999)
6. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. *J. Cryptology* 4, 3–72 (1991)
7. Bryant, R.E.: Graph-based algorithms for Boolean function manipulation. *IEEE Trans. Comput.* C-35(8), 677–691 (1986)

8. Cid, C., Huang, T., Peyrin, T., Sasaki, Y., Song, L.: Boomerang connectivity table: a new cryptanalysis tool. In: EUROCRYPT 2018. LNCS, vol. 10821, pp. 683–714. Springer (2018)
9. Daemen, J., Rijmen, V.: The Design of Rijndael: AES—The Advanced Encryption Standard. Springer (2002)
10. Fujita, M., McGeer, P.C., Yang, J.C.-Y.: Multi-terminal binary decision diagrams: an efficient data structure for matrix representation. *Formal Methods Syst. Des.* 10(2/3), 149–169 (1997)
11. Kimura, S., Clarke, E.M.: A parallel algorithm for constructing binary decision diagrams. In: ICCD 1990, pp. 220–223 (1990)
12. Knudsen, L.R.: DEAL—a 128-bit block cipher. Technical Report 151, University of Bergen (1998)
13. Lee, C.Y.: Representation of switching circuits by binary-decision programs. *Bell Syst. Tech. J.* 38(4), 985–999 (1959)
14. Matsui, M.: Linear cryptanalysis method for DES cipher. In: EUROCRYPT 1993. LNCS, vol. 765, pp. 386–397. Springer (1994)
15. Matsui, M.: On correlation between the order of S-boxes and the strength of DES. In: EUROCRYPT 1994. LNCS, vol. 950, pp. 366–375. Springer (1995)
16. Mouha, N., Wang, Q., Gu, D., Preneel, B.: Differential and linear cryptanalysis using mixed-integer linear programming. In: Inscrypt 2011. LNCS, vol. 7537, pp. 57–76. Springer (2012)
17. The Sage Developers: SageMath, the Sage Mathematics Software System. <https://www.sagemath.org> (module `sage.crypto.sboxes`)
18. Wagner, D.: The boomerang attack. In: FSE 1999. LNCS, vol. 1636, pp. 156–170. Springer (1999)